Programming Principles I

Week 2

Data Types & Conditionals

Choosing the right kind of data · Making decisions with if / else / switch

```
if (temperature > 38.0) {
    printf("Check fever");
}
```

Today’s roadmap

The goal is not to memorize every type, but to know what problem each type solves

1. Why data types exist
2. Numbers: int, float, double
3. Characters and simple text idea
4. Type conversion and traps
5. Boolean logic in C
6. if / else / else-if / switch

```c
int age = 18;
double price = 12.75;
char grade = 'A';

if (age >= 18) {
    printf("Adult ticket");
} else {
    printf("Student ticket");
}
```

By the end, students should be able to choose suitable types and write programs that behave differently for different inputs.

Why do we need data types?

A type tells C how much memory to reserve and how to interpret the bits

Student ID	GPA	Grade	Passed?
Whole number Example: 20261234	Decimal value Example: 3.74	Single symbol Example: A	Decision result true or false



The same bits can mean a number, a character, or something else. The type gives the meaning.

Guiding question: What type would you use for a student's age, tuition amount, and final letter grade?

C

Core C data types

These are enough for almost all beginner programs

Type	Use when...	Example	Format
`int`	Whole numbers	credits, quantity, age	`%d`
`float`	Decimal, lower precision	simple measurements	`%f`
`double`	Decimal, better precision	money-ish values, physics, GPA	`%lf in scanf / %f in printf`
`char`	One character	grade, menu option	`%c`

Practical rule: use int for counts, double for decimal calculations, char for one symbol.

Do not use float just because the number has a decimal. double is usually the safer default in this course.

Declaring and initializing variables

A variable should have a type, a name, and usually a clear initial value

```c
int students = 35;
double price = 4.50;
char section = 'B';

int count;        // declared, not initialized
count = 10;       // assigned later
```

Declaration creates the variable.

Initialization creates it and immediately stores the first value.

Uninitialized local variables contain garbage. Do not read them before assigning a value.

```c
int x;
printf("%d", x); // bad habit
```

Question: Which variables above are safe to print immediately?

Integer division: the first serious trap

When both operands are int, C gives an int result

```
int total = 7;
int people = 2;

printf("%d", total / people);   // 3
printf("%d", total % people);   // 1
```

7 ÷ 2 → **3 remainder 1**

Integer division removes the fractional part. It does not round.

Modulo (%) gives the remainder. Useful for even/odd, cycles, clocks, and grouping.

```
int minutes = 130;
printf("%d h %d min", minutes / 60, minutes % 60);
```

Predict: What prints for 130 minutes?

Type conversion

C sometimes converts values automatically, but we must know when precision is lost

```c
int a = 7;
int b = 2;
double result1 = a / b;
double result2 = (double)a / b;

printf("%.2f\n", result1);
printf("%.2f\n", result2);
```

result1 is 3.00 because a / b happens as integer division first.

result2 is 3.50 because casting a to double forces decimal division.

Mental model

C calculates the expression first, then stores the result into the variable.

Guiding question: Why is “double result = a / b;” still not enough?

C

Characters: char

A char stores one character, written with single quotes

```
char grade = 'A';
char answer = 'y';
char digit = '7';

printf("%c\n", grade);
```

Use single quotes for one character: 'A'

Do not confuse character 7 with number 7.

```
char digit = '7';
int number = 7;
```

Question: Which one can be used directly in arithmetic?

Important: strings come next week. Today char means one symbol only.

Comparisons produce yes/no answers

C uses numbers for truth: 0 is false, non-zero is true

`>`	greater than
`>=`	greater or equal
`<`	less than
`<=`	less or equal
`==`	equal to
`!=`	not equal

```
int age = 19;
printf("%d", age >= 18);   // 1
printf("%d", age == 18);   // 0
```

Warning: = assigns. == compares.

Predict: What is the value of 5 != 3 ?

Combining conditions

Real decisions usually depend on more than one condition

```c
int age = 20;
int hasCard = 1;

if (age >= 18 && hasCard) {
    printf("Allowed");
}
```

&&

AND
Both sides must be true

||

OR
At least one side is true

!

NOT
Reverses the truth value

Real-life example: “Can enter exam” = registered AND not late AND has ID card.

Guiding question: Write the condition for a discount: student OR age under 18.

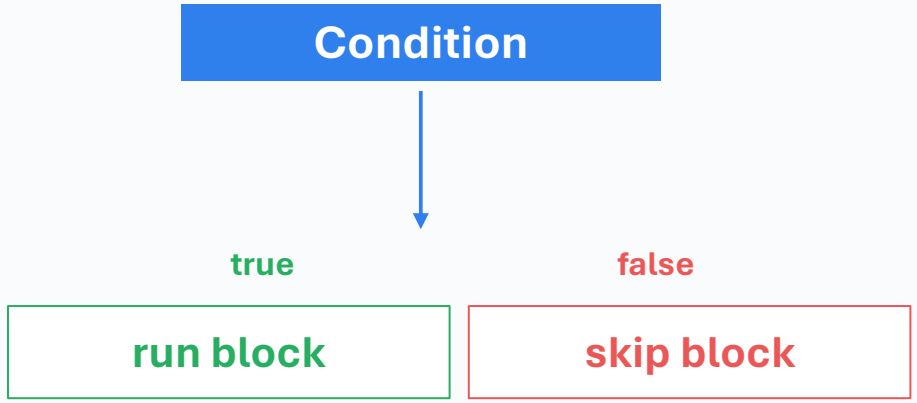
The if statement

An if statement runs a block only when the condition is true

```c
int score;
printf("Score: ");
scanf("%d", &score);

if (score >= 50) {
    printf("Passed\n");
}

printf("Done\n");
```



Question: If score is 49, which lines print?

Question: If score is 50, which lines print?

if / else

Use else when exactly one of two paths should run

```c
int age;
printf("Age: ");
scanf("%d", &age);

if (age >= 18) {
    printf("Adult ticket\n");
} else {
    printf("Student ticket\n");
}
```

If the condition is true, the if block runs.

If it is false, the else block runs.

Never both. Never neither.

Mini-task: Modify the program: age < 6 gets “free ticket”, otherwise normal ticket.

else-if chains

Use else-if when there are multiple categories

```c
int score;
scanf("%d", &score);

if (score >= 90) {
    printf("A");
} else if (score >= 80) {
    printf("B");
} else if (score >= 70) {
    printf("C");
} else {
    printf("Needs improvement");
}
```

C checks from top to bottom.

The first true condition wins.

Order matters. Put the strongest condition first in this example.

Predict: What prints for 85? What about 95? What about 69?

Order can break your logic

A program can compile successfully and still be logically wrong

```
if (score >= 70) {
    printf("C");
} else if (score >= 80) {
    printf("B");
} else if (score >= 90) {
    printf("A");
}
```

This code compiles, but A and B are unreachable.

For score = 95, the first condition score >= 70 is already true.

Fix: check 90 first, then 80, then 70.

Class question: Give another real-life situation where order matters.

Nested decisions

Sometimes a second decision only makes sense inside the first decision

```c
int loggedIn = 1;
int isAdmin = 0;

if (loggedIn) {
    if (isAdmin) {
        printf("Admin panel");
    } else {
        printf("User page");
    }
} else {
    printf("Please log in");
}
```

Real-life idea: first check if the person has entered the building, then check which room they may enter.

Nested if is useful, but too much nesting makes code hard to read.

Guiding question: What prints when loggedIn = 1 and isAdmin = 0?

switch statements

switch is useful when one variable is compared to fixed values

```c
char option;
printf("A/B/C: ");
scanf(" %c", &option);

switch (option) {
    case 'A':
        printf("Add item");
        break;
    case 'B':
        printf("View cart");
        break;
    default:
        printf("Unknown option");
}
```

Good for menus and fixed choices.

break stops the program from falling into the next case.

The space before %c in scanf skips leftover whitespace.

Question: What happens if the user enters D?

Common Week 2 bugs

Most beginner bugs here come from small symbols with big meanings

= instead of ==

```
if (x = 5)
```

Integer division

```
double avg = sum / count
```

Missing braces

only one statement belongs to if

Wrong else-if order

lower threshold first

Wrong format specifier

%d for double input

Forgetting break

switch falls through

Debugging habit: when the output is wrong, trace with one specific input instead of staring at the whole program.

Worked example: BMI category

A real program combines data types, input, calculations, and decisions

Problem

Read weight in kg and height in meters. Compute BMI = weight / height^2 and print a simple category.

Question: Why do we not write bmi >= 18.5 && bmi < 25.0 for the second condition?

Answer idea: previous failed conditions already give information.

```c
double weight, height, bmi;
scanf("%lf %lf", &weight, &height);

bmi = weight / (height * height);

if (bmi < 18.5) {
    printf("Underweight");
} else if (bmi < 25.0) {
    printf("Normal");
} else if (bmi < 30.0) {
    printf("Overweight");
} else {
    printf("Obese");
}
```

Worked example: electricity bill

Tiered pricing is a strong real-life example of conditionals

Rule

First 100 kWh: 0.10 per kWh
Next 100 kWh: 0.15 per kWh
Above 200 kWh: 0.20 per kWh

Predict the bill for 80, 150, and 250 kWh.

```c
double kwh, bill;
scanf("%lf", &kwh);

if (kwh <= 100) {
    bill = kwh * 0.10;
} else if (kwh <= 200) {
    bill = 100 * 0.10 + (kwh - 100) * 0.15;
} else {
    bill = 100 * 0.10 + 100 * 0.15 + (kwh - 200) * 0.20;
}

printf("Bill = %.2f", bill);
```

Code tracing exercise

Before writing code, students must be able to read code

```c
int x = 5;
int y = 12;

if (x * 2 > y) {
    printf("A");
} else if (y % x == 2) {
    printf("B");
} else {
    printf("C");
}

printf("D");
```

Questions

1. What is x * 2 > y?
2. What is y % x?
3. What exactly is printed?
4. Which branch runs?

Do not run it first. Trace it like an exam question.

C

In-class practice tasks

Start simple, then combine ideas

Task 1

Read two integers and print the larger one.

Task 2

Read an integer and print whether it is even or odd.

Task 3

Read a score and print A/B/C/D/F using else-if.

Task 4

Build a tiny menu with switch: add, subtract, multiply.

Task 5

Read hours and hourly rate. Include overtime after 40 hours.

Guidance: For each task, first write inputs, outputs, and conditions in plain English.

C

Quiz-style questions

Short questions, but they reveal real understanding

```c
int a = 9, b = 4;
printf("%d %d", a / b, a % b);
```

Q1: What is printed?

```c
double avg = 7 / 2;
printf("%.1f", avg);
```

Q2: Why is this not 3.5?

```c
if (x >= 0)
    printf("positive");
    printf("done");
```

Q3: Which line is always printed?

Good answer format: do not only give the result; explain the rule that produced it.

Week 2 summary

What students should take away before Week 3

Data types	choose how values are stored and interpreted	**Comparisons**	produce true/false values represented by numbers
int vs double	whole-number operations behave differently from decimal operations	**Conditionals**	make programs choose between paths
char	stores one symbol, not a word	**Tracing**	is the safest way to understand unfamiliar code

```
// Always ask:
// 1. What data do I need?
// 2. What type should store it?
// 3. What decisions must the program make?
```